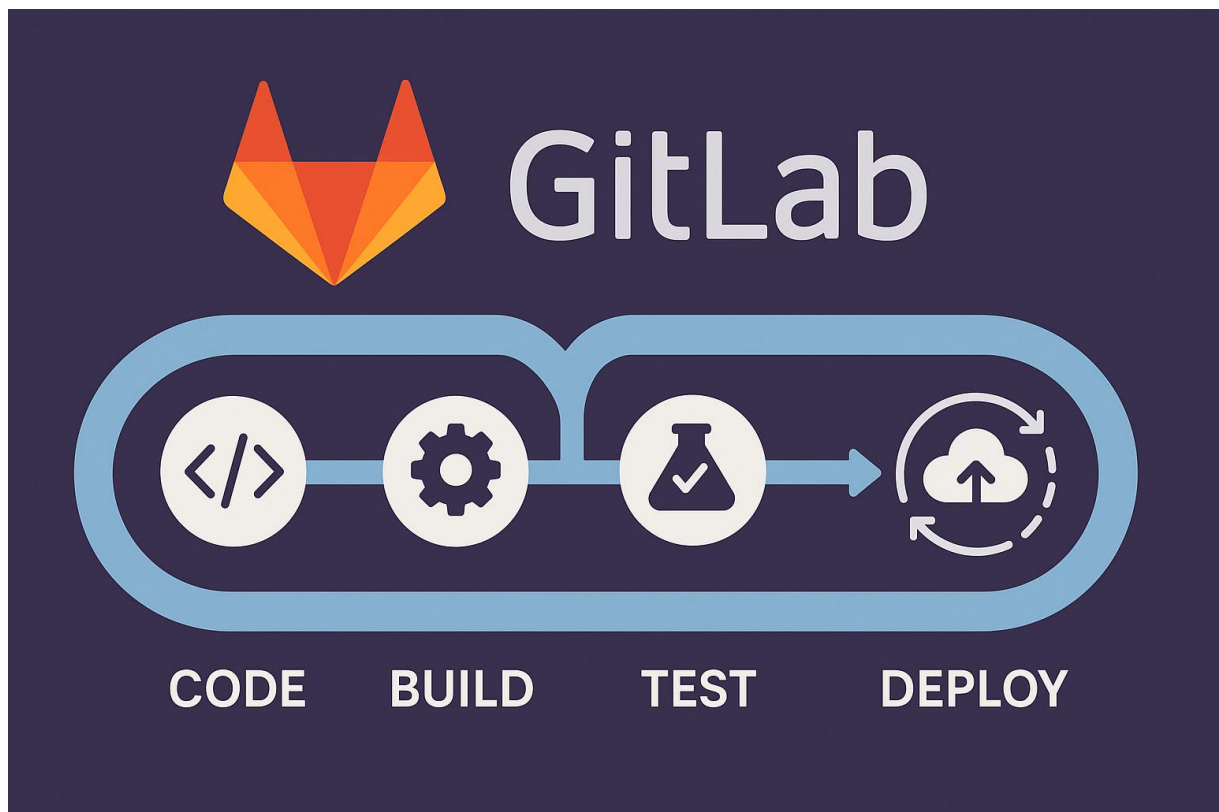


TP : CI/CD



Fait par :

Ulrich Noéjules OGO NDAYEGUIA

Abdoulaye Diallo

Contexte du TP

Dans cette série de TP CI-CD nous allons voir comment mettre en place une pipeline CI-CD en utilisant Gitlab.

Gitlab est une **plateforme DevOps complète** qui permet de gérer tout le cycle de vie du développement logiciel, de la planification à la mise en production, en passant par l'intégration et le déploiement continu (CI/CD).

CI/CD (Intégration Continue / Déploiement Continu ou Livraison Continue) est d'**automatiser et accélérer le processus de développement logiciel**, tout en garantissant la qualité du code et la fiabilité des livraisons.

Voici les objectifs principaux :

1. Détection rapide des erreurs

- Grâce à l'intégration continue (CI), chaque modification du code est automatiquement testée.
- Cela permet de repérer les bugs rapidement, dès qu'ils sont introduits.

2. Livraisons plus rapides

- Avec la livraison continue ou le déploiement continu (CD), les nouvelles fonctionnalités peuvent être mises en production plus fréquemment et de manière fiable.
- Le déploiement devient un processus régulier et non un événement stressant.

3. Automatisation du cycle de développement

- Les tests, les compilations, et les déploiements sont automatisés.
- Moins d'intervention humaine = moins d'erreurs manuelles.

4. Amélioration de la qualité

- Les pipelines CI/CD intègrent des tests unitaires, fonctionnels, et parfois des analyses de sécurité ou de performance.
- Cela permet de maintenir un haut niveau de qualité à chaque version.

5. Réduction des délais de mise en marché

- Une équipe peut livrer plus rapidement les nouvelles versions d'une application, ce qui est un gros avantage compétitif.

Installation de Docker sur notre serveur Debian 11

```
root@testing:~# apt install -y docker.io
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
  cgroupfs-mount containerd git git-man iptables liberror-perl libintl-perl libintl-xs-perl libip6tc2 libmodule-find-perl
  libnetfilter-contrack3 libnfnetlink0 libproc-processtable-perl libsort-naturally-perl needrestart runc tini
Paquets suggérés :
  containernetworking-plugins docker-doc aufs-tools btrfs-progs deboststrap rinse rootlesskit xfsprogs zfs-fuse | zfsutils-linux
  git-daemon-run | git-daemon-sysvinit git-doc git-el git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn firewallld
  needrestart-session | libnotify-bin iucode-tool
Paquets recommandés :
  criu
Les NOUVEAUX paquets suivants seront installés :
  cgroupfs-mount containerd docker.io git git-man iptables liberror-perl libintl-perl libintl-xs-perl libip6tc2 libmodule-find-perl
  libnetfilter-contrack3 libnfnetlink0 libproc-processtable-perl libsort-naturally-perl needrestart runc tini
0 mis à jour, 18 nouvellement installés, 0 à enlever et 17 non mis à jour.
Il est nécessaire de prendre 69,2 Mo dans les archives.
Après cette opération, 311 Mo d'espace disque supplémentaires seront utilisés.
Réception de :1 http://security.debian.org/debian-security bullseye-security/main amd64 docker.io amd64 20.10.5+dfsg1-1+deb11u4 [36,8 M
B]
Réception de :2 http://deb.debian.org/debian bullseye/main amd64 runc amd64 1.0.0-rc93+ds1-5+deb11u5 [2 431 kB]
Réception de :3 http://deb.debian.org/debian bullseye/main amd64 containerd amd64 1.4.13-ds1-1-deb11u4 [20,9 MB]
Réception de :4 http://deb.debian.org/debian bullseye/main amd64 libip6tc2 amd64 1.8.7-1 [35,0 kB]
Réception de :5 http://deb.debian.org/debian bullseye/main amd64 libnfnetlink0 amd64 1.0.1-3+b1 [13,9 kB]
Réception de :6 http://deb.debian.org/debian bullseye/main amd64 libnetfilter-contrack3 amd64 1.0.8-3 [40,6 kB]
Réception de :7 http://deb.debian.org/debian bullseye/main amd64 iptables amd64 1.8.7-1 [382 kB]
Réception de :8 http://deb.debian.org/debian bullseye/main amd64 tini amd64 0.19.0-1 [255 kB]
Réception de :9 http://security.debian.org/debian-security bullseye-security/main amd64 git-man all 1:2.30.2-1+deb11u4 [1 831 kB]
Réception de :10 http://deb.debian.org/debian bullseye/main amd64 cgroupfs-mount all 1.4 [6 276 B]
Réception de :11 http://deb.debian.org/debian bullseye/main amd64 liberror-perl all 0.17029-1 [31,0 kB]
Réception de :12 http://deb.debian.org/debian bullseye/main amd64 libintl-perl all 1.26-3+deb11u1 [742 kB]
Réception de :13 http://deb.debian.org/debian bullseye/main amd64 libintl-xs-perl amd64 1.26-3+deb11u1 [40,7 kB]
Réception de :14 http://deb.debian.org/debian bullseye/main amd64 libmodule-find-perl all 0.15-1 [11,0 kB]
Réception de :15 http://deb.debian.org/debian bullseye/main amd64 libproc-processtable-perl amd64 0.59-2+b1 [45,9 kB]
Réception de :16 http://deb.debian.org/debian bullseye/main amd64 libsort-naturally-perl all 1.03-2 [13,5 kB]
Réception de :17 http://security.debian.org/debian-security bullseye-security/main amd64 git amd64 1:2.30.2-1+deb11u4 [5 566 kB]
94% [17 git 2 170 kB/5 566 kB 39%] 5 061 kB/s 0s1s
```

Installation de docker :

apt install-y docker.io

systemctl enable--now docker

Vérifions la version de docker installée : docker--version

Ajouter le dépôt officiel GitLab Runner :

curl-L https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh | sudo bash

Téléchargement :

wget https://gitlab-runner-downloads.s3.amazonaws.com/latest/binaries/gitlab-runner-linux-amd64

mv gitlab-runner-linux-amd64 /usr/local/bin/gitlab-runner

chmod +x /usr/local/bin/gitlab-runner

Enfin, crée un lien symbolique pour être sûr que la commande est accessible partout :

ln-s /usr/local/bin/gitlab-runner /usr/bin/gitlab-runner

Créer un user git-lab :

useradd--comment 'GitLab Runner'--create-home gitlab-runner--shell /bin/bash

Installer gitlab :

gitlab-runner install--user=gitlab-runner--working-directory=/home/gitlab-runner

gitlab-runner start

gitlab-runner status

```

root@testing:~# gitlab-runner restart
Runtime platform                                arch=amd64 os=linux pid=930 revision=0f67ff19 version=17.11.0
root@testing:~#
root@testing:~# gitlab-runner status
Runtime platform                                arch=amd64 os=linux pid=1402 revision=0f67ff19 version=17.11.0
gitlab-runner: Service is running
root@testing:~#
root@testing:~# systemctl enable --now gitlab-runner
root@testing:~#
root@testing:~#

```

gitlab-runner register  Entrer

--url => https://gitlab.com

--token => glrt-r5GDvoy08L-cpbZqF8XBEW86MQpwOjE1ZGpjZQp00jMKdTpmDHJhZRg.01.1j1gru6sy

Enter a name of runner : project

Enter an executor : docker

Enter the default Docker image () : docker : stable

```

root@testing:~#
root@testing:~# gitlab-runner register
Runtime platform                                arch=amd64 os=linux pid=4554 revision=0f67ff19 version=17.11.0
Running in system-mode.

Enter the GitLab instance URL (for example, https://gitlab.com/):
https://gitlab.com
Enter the registration token:
glrt-r5GDvoy08L-cpbZqF8XBEW86MQpwOjE1ZGpjZQp00jMKdTpmDHJhZRg.01.1j1gru6sy
Verifying runner... is valid
Enter a name for the runner. This is stored only in the local config.toml file:
[testing]: project
Enter an executor: virtualbox, docker, instance, shell, ssh, parallels, kubernetes, docker-autoscaler, custom, docker-windows, docker+m
achine:
docker
Enter the default Docker image (for example, ruby:2.7):
docker:stable
Runner registered successfully. Feel free to start it, but if it's running already the config should be automatically reloaded!
Configuration (with the authentication token) was saved in "/etc/gitlab-runner/config.toml"
root@testing:~#

```

Assignment - 1 | GitLab CI Base Yml

Création du Projet et du dépôt GitLAB :

Créer un nouveau projet



Créer un projet vide
Créez un projet vide pour, entre autres, stocker vos fichiers, planifier votre travail et collaborer sur le code.



Créer à partir d'un modèle
Créez un projet pré-rempli avec les fichiers nécessaires pour vous permettre de démarrer rapidement.



Importer un projet
Migrez vos données depuis une source externe comme GitHub, Bitbucket ou une autre instance de GitLab.



Exécuter CI/CD pour un dépôt externe
Connecter votre dépôt externe à GitLab CI/CD.

[Vous pouvez également créer un projet en ligne de commande. Afficher la commande](#)

Créer un projet vide

Créez un projet vide pour, entre autres, stocker vos fichiers, planifier votre travail et collaborer sur le code.

Nom du projet

Doit commencer par une lettre minuscule ou majuscule, un chiffre, un emoji ou un tiret bas. Peut également contenir des points, des signes plus, des tirets ou des espaces.

URL du projet

Identifiant « slug » du projet

Cible du déploiement du projet (facultatif)

Niveau de visibilité 

☐ Privé
L'accès au projet doit être explicitement accordé à chaque utilisateur. Si ce projet fait partie d'un groupe, l'accès est accordé aux membres du groupe.

☐ Interne 
Le projet est accessible à tout utilisateur connecté, à l'exception des utilisateurs externes.

☒ Public
Le projet est accessible sans aucune authentification.

Configuration du Projet

☒ Initialiser le dépôt avec un README
Vous permet de cloner immédiatement le dépôt de ce projet. Si vous prévoyez de pousser un dépôt existant, vous pouvez ignorer cette option.

☐ Activer les tests statiques de sécurité des applications (SAST)
Analyser votre code source à la recherche de failles de sécurité connues. En savoir plus.

[> Paramètres expérimentaux](#)



README.md5,97 Kio

CodeAperçu

project

Getting started

To make it easy for you to get started with GitLab, here's a list of recommended next steps.

Already a pro? Just edit this README.md and make it your own. Want to make it easy? [Use the template at the bottom!](#)

Add your files

☐ Create or upload files

☐ Add files using the command line or push an existing Git repository with the following command:

```
cd existing_repo
git remote add origin https://gitlab.com/ulrichogo/project.git
git branch -M main
git push -uf origin main
```

Integrate with your tools

Création du Runner : Paramètres => CI/CD => Runners

Ulrich OGO / project / Paramètres CI/CD / Nouveau runner

Nouvel runner du projet

Créez un runner de projet pour générer une commande qui enregistre le runner avec toutes ses configurations.

Étiquettes

Ajouter des labels pour spécifier les jobs que le runner peut exécuter. En savoir plus.

project

Séparez les étiquettes avec des virgules. Par exemple, `macos, shared`.

☐ Exécuter les jobs sans étiquettes

Utilisez le runner pour les jobs sans étiquette en plus des jobs étiquetés.

Configuration (facultatif)

Description du runner

project

☐ En pause

Empêcher le runner d'accepter de nouveaux jobs.

☐ Protégé

Utiliser le runner sur les pipelines pour les branches protégées uniquement.

☐ Verrouiller aux projets en cours

Utiliser le runner uniquement sur les projets qui lui sont actuellement assignés. Seuls les administrateurs peuvent modifier les projets assignés.

Durée maximale d'exécution du job

Durée maximale pendant laquelle le runner peut s'exécuter avant de prendre fin. Si un projet a un délai d'attente plus court avant expiration des jobs, celui du runner de l'instance est utilisé à la place.

Entrer le délai d'attente du job en secondes. Doit être d'au moins 600 secondes.

Créer un runner

Lançons notre premier code CI/CD

The screenshot displays the GitLab CI/CD interface. At the top, the breadcrumb navigation shows 'Ulrich OGO / project / Dépôt'. Below this, the file '.gitlab-ci.yml' is selected, and the interface shows the configuration for 'Assignment - 1 | GitLab CI Base Yml'. A message indicates that the configuration is valid. The code for the pipeline is shown in a code editor, defining a 'test' stage with a 'test_job'.

```
1 stages:
2   - test
3
4 test_job:
5   stage: test
6   tags:
7     - project # tags pour le runner
8   script:
9     - echo "Configuration définie pour un test simple."
10    - echo "Pour vérifier que le pipeline fonctionne correctement !"
```

Below the code editor, the 'Pipeline' tab is selected, showing the execution history. The pipeline is shown as 'Succès' (Success) and 'Ulrich OGO a créé le pipeline pour la validation 309cc344'. The pipeline consists of one job, 'test', which is shown as 'Succès' (Success).

Assignment - 2 | GitLab CI Stages

Dans cette partie nous allons mettre en jour le pipeline avec d'autres job notamment :

✓ pre_check_job ✓ build_job ✓ test_job Configurez trois tâches: pre_check_job, build_job et test_job. Affectez pre_check_job à l'étape de pré-check, build_job à l'étape de build et test_job à l'étape de test. Dans la section script de pre_check_job, ajoutez- echo « Vérifications en cours » Dans la section script de build_job, ajoutez- echo "Building"

Catalogue CI/CD
Aide

```

1  stages:
2    - pre_check
3    - build
4    - test
5
6  pre_check_job:
7    stage: pre_check
8    script:
9      - echo "Vérifications en cours "
10
11 build_job:
12   stage: build
13   script:
14     - echo "Building"
15
16 test_job:
17   stage: test
18   script:
19     - echo "Tester mon code"

```

Message de commit
Assignment - 2 | GitLab CI Stages

Branche
main

Valider les modifications
Réinitialiser

Mettre à jour le fichier .gitlab-ci.yml
Supprimer

Réussi Ulrich OGO a créé le pipeline pour la validation @f247c94 il y a 2 minutes, terminé il y a 1 minute

Pour main

dernier branche 3 jobs 1.44 1 minute 25 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs 3 Tests 0

```

graph LR
    subgraph pre_check
        pre_check_job[pre_check_job]
    end
    subgraph build
        build_job[build_job]
    end
    subgraph test
        test_job[test_job]
    end
    pre_check --> build
    build --> test

```

Assignment - 3 | GitLab CI Jobs and Scripts

Consignes : Voici les modifications à apporter à votre pipeline CI GitLab :

- Mettez à jour le script du `pre_check_job` pour répertorier tous les conteneurs Docker en cours d'exécution. (docker ps)
- Mettez à jour le script du `pre_check_job` pour répertorier toutes les images Docker. (docker images)

NB : pour exécuter les commandes nous allons modifier le fichier : `vim /etc/gitlab-runner/config.toml` **privileged = true** pour permettre l'exécution des commandes

```
concurrent = 1
check_interval = 0
connection_max_age = "15m0s"
shutdown_timeout = 0

[session_server]
session_timeout = 1800

[[runners]]
name = "project"
url = "https://gitlab.com"
id = 47244233
token = "glrt-r5Gpvoy08L-cpbzqF8XBEW86MQpw0jE1ZGpjZqp00jMkdTpmDHjHzRg.01.1j1qru6sy"
token_obtained_at = 2025-05-02T02:10:08Z
token_expires_at = 0001-01-01T00:00:00Z
executor = "docker"
[runners.cache]
MaxuploadedArchivesize = 0
[runners.cache.s3]
[runners.cache.gcs]
[runners.cache.azure]
[runners.docker]
tls_verify = false
image = "docker:stable"
privileged = true
disable_entrypoint_overwrite = false
oom_kill_disable = false
disable_cache = false
volumes = ["/cache"]
shm_size = 0
network_mtu = 0
~
~
~
~
~
~
~
~
~
~
```

25,21

Tout

Catalogue CI/CD Aide

```
1 stages:
2   - pre_check
3   - build
4   - test
5
6 pre_check_job:
7   stage: pre_check
8   image: docker:stable
9   services:
10    - docker:dind
11   script:
12     - echo "Conteneur Docker en cours d'exécution"
13     - docker ps
14     - echo "Liste de toutes les images Docker disponibles ::"
15     - docker images
16
17 build_job:
18   stage: build
19   script:
20     - echo "Construction du projet en cours..."
21
22 test_job:
23   stage: test
24   script:
25     - echo "Exécution des tests automatisés"
```

Message de commit

Assignment - 3 | GitLab CI Jobs and Scripts

Branche

main

Valider les modifications

Reinitialiser

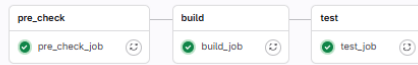
Assignment - 3 | GitLab CI Jobs and Scripts

✓ **Reussir** Ulrich OGO a créé le pipeline pour la validation `f67a6643` Il y a 2 minutes, terminé il y a 47 secondes

Pour **main**

✓ **dernier** **branche** 3 jobs 1.64 1 minute 38 secondes, mis en file d'attente pendant 0 secondes

Pipeline Jobs 3 Tests 0



Assignment - 4 | GitLab CI | DockerFile Setup

Créez un nouveau fichier dans votre référentiel au niveau du chemin racine et nommez-le Dockerfile. Ajoutez le contenu ci-dessous dans le fichier. 1. FROM debian 2. LABEL name="devops" 3. CMD ["sleep", "120"]

Ulrich OGO / project / Dépôt

Nouveau fichier

Annuler Valider les modifications

main DockerFile

```
1 FROM debian
2 LABEL name="devops"
3 CMD ["sleep", "120"]
4
```

Pas de retour à la ligne

Ulrich OGO / project / Pipelines / #1796781015

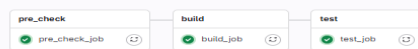
DockerFile

✓ **Reussir** Ulrich OGO a créé le pipeline pour la validation `36261866` Il y a 2 minutes, terminé il y a 1 minute

Pour **main**

✓ **dernier** **branche** 3 jobs 1.63 1 minute 37 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs 3 Tests 0



Assignment - 5 | GitLab CI Jobs and Scripts

Configurez le pipeline CI GitLab pour les modifications ci-dessous : Mettez à jour le script de tâche `build_job` pour créer l'image Docker à l'aide de Dockerfile, que nous avons téléchargé dans la dernière tâche. Et Conservez la balise la plus récente (par défaut)

Ulrich OGO / project / Éditeur de pipeline

Catalogue CI/CD Aide

```
1 stages:
2   - pre_check
3   - build
4   - test
5
6 pre_check_job:
7   stage: pre_check
8   image: docker:stable
9   services:
10  - docker:dind
11  script:
12    - echo "Conteneur Docker en cours d'exécution"
13    - docker ps
14    - echo "Liste de toutes Les images Docker disponibles :"

Message de commit



Assignment - 5 | GitLab CI Jobs and Scripts



Branche



main



Valider les modifications Reinitialiser


```

Ulrich OGO / project / Pipelines / #1796830035

Mettre à jour le fichier .gitlab-ci.yml

Reussir Ulrich OGO a créé le pipeline pour la validation 2158f3b8 Il y a 4 minutes, terminé Il y a 2 minutes

Pour main

dernier branche eo 3 jobs 1.87 1 minute 52 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs Tests

pre_check build test

pre_check_job build_job test_job

```
26 $ docker build -t non-image:latest .
27 ## building with "default" instance using docker driver
28 #1 [internal] load build definition from Dockerfile
29 #1 transferring dockerfile: 93B done
30 #1 DONE 0.0s
31 #2 [internal] load metadata for docker.io/library/debian:latest
32 #2 DONE 0.5s
33 #3 [internal] load .dockerignore
34 #3 transferring context: 2B done
35 #3 DONE 0.0s
36 #4 [1/1] FROM docker.io/library/debian:latest@sha256:264982ff4d1800efa74540837e2c43ca5137a53a83f8f62c7b3803c0f8bdc56
37 #4 resolve docker.io/library/debian:latest@sha256:264982ff4d1800efa74540837e2c43ca5137a53a83f8f62c7b3803c0f8bdc56 8.52kB / 8.52kB done
38 #4 sha256:264982ff4d1800efa74540837e2c43ca5137a53a83f8f62c7b3803c0f8bdc56 8.52kB / 8.52kB done
39 #4 sha256:6bc38d909563f3868edd6689e29eb3fb284ab8affce8d389f332fc91c2dd91 1.62kB / 1.62kB done
40 #4 sha256:b2ab84c087f8ea81d95c535844a7a5eea4692a79cb40f08b0c3fe08cb4626 453B / 453B done
41 #4 sha256:c095a52c82353f0b2b6f9be8549ac916c3fb1dc8d4bacd405eac7f28562ec9f2 8B / 48.49MB 0.1s
42 #4 sha256:c095a52c82353f0b2b6f9be8549ac916c3fb1dc8d4bacd405eac7f28562ec9f2 16.78MB / 48.49MB 0.3s
43 #4 sha256:c095a52c82353f0b2b6f9be8549ac916c3fb1dc8d4bacd405eac7f28562ec9f2 39.85MB / 48.49MB 0.4s
44 #4 sha256:c095a52c82353f0b2b6f9be8549ac916c3fb1dc8d4bacd405eac7f28562ec9f2 48.49MB / 48.49MB 0.6s
45 #4 sha256:c095a52c82353f0b2b6f9be8549ac916c3fb1dc8d4bacd405eac7f28562ec9f2 48.49MB / 48.49MB 0.7s done
46 #4 extracting sha256:c095a52c82353f0b2b6f9be8549ac916c3fb1dc8d4bacd405eac7f28562ec9f2
47 #4 extracting sha256:c095a52c82353f0b2b6f9be8549ac916c3fb1dc8d4bacd405eac7f28562ec9f2 2.8s done
48 #4 DONE 2.8s
49 #5 exporting to image
50 #5 exporting layers done
51 #5 writing image sha256:f2b269f37ead4669ee26496f8770f6c275274601b8669f4f9ed91ad8537565b3 done
52 #5 naming to docker.io/library/non-image:latest done
53 #5 DONE 0.0s
54 $ echo "Image construite avec succès"
55 Image construite avec succès
56 Cleaning up project directory and file based variables
57 Job succeeded
```

Assignment - 6 | GitLab CI Jobs and Scripts

Configurez le pipeline CI GitLab pour les modifications ci-dessous :

1. Mettre à jour le script `test_job` pour afficher tous les conteneurs Docker en cours d'exécution et arrêtés avec la commande `docker ps -a`

```
26 test_job:
27   stage: test
28   image: docker:latest
29   services:
30     - docker:dind
31   script:
32     - echo "Afficher tous les conteneur "
33     - docker ps -a
34
```

Ulrich OGO / project / Pipelines / #1798845806

Assignment - 6 | GitLab CI Jobs and Scripts

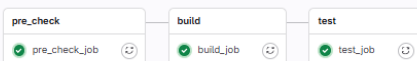
Supprimer

✓ Réussi Ulrich OGO a créé le pipeline pour la validation c6798530. Il y a 4 minutes, terminé il y a 1 minute

Pour main

dernier branche 40 3 jobs 2.05 2 minutes 3 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs 3 Tests 0



```
1 Running with gitlab-runner 17.10.0-pre.41.g5c23fd8e (5c23fd8e)
2 on blue-4.saas-linux-small-amd64.runners-manager.gitlab.com/default J2nyww-s, system ID: s_cf1798852952
3 Preparing the "docker-machine" executor
4 Using Docker executor with image docker:latest ...
5 Starting service docker:dind...
6 Pulling docker image docker:dind ...
7 Using docker image sha256:721f8c2d35917e067eee791d06c09d565d2c4b6b95aa7df8190c04b56d3de9e0 for docker:dind with digest docker@sha256:f49e1c71b5d9f8abe53715f78996ce42b8be4
b1ec03875d187dfe3c03de1dc00 ...
8 Waiting for services to be up and running (timeout 30 seconds)...
9 Pulling docker image docker:latest ...
10 Using docker image sha256:721f8c2d35917e067eee791d06c09d565d2c4b6b95aa7df8190c04b56d3de9e0 for docker:latest with digest docker@sha256:f49e1c71b5d9f8abe53715f78996ce42b8b
e4b1ec03875d187dfe3c03de1dc00 ...
11 Preparing environment
12 Running on runner-j2nyww-s-project-69495854-concurrent-0 via runner-j2nyww-s-l-s-amd64-1746162013-34ccf8f6...
13 Getting source from Git repository
14 Fetching changes with git depth set to 20...
15 Initialized empty Git repository in /builds/ulrichogo/project/.git/
16 Created fresh repository.
17 Checking out c6798530 as detached HEAD (ref is main)...
18 Skipping Git submodule setup
19 $ git remote set-url origin "${CI_REPOSITORY_URL}" || echo 'Not a git repository; skipping'
20 Executing "step_script" stage of the job script
21 Using docker image sha256:721f8c2d35917e067eee791d06c09d565d2c4b6b95aa7df8190c04b56d3de9e0 for docker:latest with digest docker@sha256:f49e1c71b5d9f8abe53715f78996ce42b8b
e4b1ec03875d187dfe3c03de1dc00 ...
22 $ echo "Afficher tous les conteneurs"
23 Afficher tous les conteneurs
24 $ docker ps -a
25 CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
26 Cleaning up project directory and file based variables
27 Job succeeded
```

Assignment - 7 | GitLab CI User Defined Variables

Configuration du pipeline CI GitLab pour les modifications ci-dessous Ajout d'une variable nommée `image_Name` avec la valeur `devops-image` dans les paramètres de la tâche `build_job`. Et mettre à jour le script de `build_job` pour utiliser la variable définie « Image Name » comme nom de notre image Docker.

Catalogue CI/CD

Aide

```
3  - build
4  - test
5
6  pre_check_job:
7    stage: pre-check
8    image: docker:stable
9    services:
10     - docker:dind
11    script:
12     - echo "Conteneur Docker en cours d'exécution"
13     - docker ps
14     - echo "Liste de toutes les images Docker disponibles :"

Message de commit



Assignment - 7 | GitLab CI User Defined Variables



Branche



main



Valider les modifications Réinitialiser


```

Assignment - 7 | GitLab CI User Defined Variables

Reussit Ulrich OGO a créé le pipeline pour la validation 35465035 Il y a 3 minutes, terminé il y a 1 minute

Pour main

demier branche 40 3 jobs 1.99 1 minute 59 secondes, mis en file d'attente pendant 0 secondes

Pipeline Jobs Tests

pre-check

build

test

pre_check_job

build_job

test_job

```
23 $ echo "Construction de l'image Docker"
24 $ docker build -t $IMAGE_NAME .
25
26 #0 building with "default" instance using docker driver
27 #1 [internal] load build definition from Dockerfile
28 #1 transferring dockerfile: 93B done
29 #1 DONE 0.0s
30 #2 [internal] load metadata for docker.io/library/debian:latest
31 #2 DONE 0.4s
32 #3 [internal] load .dockerignore
33 #3 transferring context: 2B done
34 #3 DONE 0.0s
35 #4 [1/1] FROM docker.io/library/debian:latest@sha256:264982ff4d18080fa74540837e2c43ce5137a53e3f8f62c7b3883c0f8bdc56
36 #4 resolve docker.io/library/debian:latest@sha256:264982ff4d18080fa74540837e2c43ce5137a53e3f8f62c7b3883c0f8bdc56 0.0s done
37 #4 sha256:264982ff4d18080fa74540837e2c43ce5137a53e3f8f62c7b3883c0f8bdc56 8.52kB / 8.52kB done
38 #4 sha256:6bc3d999583f38680add6689e29eb3fb284ab8affce8d0389f332fc91c2dd91 1.02kB / 1.02kB done
39 #4 sha256:b2ab84c087feae81d95c5358d44ad7a54ea4693a79cb40fb05bd3fe08cb4d426 453B / 453B done
40 #4 sha256:cf95a52c02353f8b2b6f9be0549ac916c3fb1dc8d4bacd405eac7f28562ec9f2 0B / 48.49MB 0.1s
41 #4 sha256:cf95a52c02353f8b2b6f9be0549ac916c3fb1dc8d4bacd405eac7f28562ec9f2 23.07MB / 48.49MB 0.3s
42 #4 sha256:cf95a52c02353f8b2b6f9be0549ac916c3fb1dc8d4bacd405eac7f28562ec9f2 46.14MB / 48.49MB 0.4s
43 #4 sha256:cf95a52c02353f8b2b6f9be0549ac916c3fb1dc8d4bacd405eac7f28562ec9f2 48.49MB / 48.49MB 0.6s done
44 #4 extracting sha256:cf95a52c02353f8b2b6f9be0549ac916c3fb1dc8d4bacd405eac7f28562ec9f2 0.1s
45 #4 DONE 2.7s
46 #5 exporting to image
47 #5 exporting layers done
48 #5 writing image sha256:f2b269f37ead4660ee26496f8778f6e275274681b8660f4ffed91ad8537565b3 done
49 #5 naming to docker.io/library/devops-image done
50 #5 DONE 0.0s
51 $ echo "Image construite avec succès"
52 Image construite avec succès
53 Cleaning up project directory and file based variables
54 Job succeeded
```

Assignment - 8 | GitLab CI Project or Group Level Variables

Configuration du pipeline CI GitLab pour les modifications ci-dessous Ajoutez une variable GitLab nommée « Image_Name » avec la valeur « devops » dans la section Setting-> CI/CD Variables

The screenshot shows the GitLab CI/CD Parameters page for a project named 'Ulrich OGO'. The left sidebar contains a navigation menu with the following items: 'Projet', 'Général', 'Intégrations', 'Crochets web', 'Jetons d'accès', 'Dépôt', 'Requêtes de fusion', 'CI/CD', 'Paquets et registres', 'Supervision', and 'Quotas d'utilisation'. The 'CI/CD' item is highlighted. The main content area is titled 'Variables' and contains the following sections:

- Rôle minimum autorisé à utiliser les variables de pipeline**: Sélectionnez le rôle minimum autorisé à exécuter un nouveau pipeline avec des variables. Qu'est-ce qu'une variable de pipeline ?
 - ☒ Aucun rôle autorisé: Les variables de pipeline ne peuvent pas être utilisées.
 - ☐ Propriétaire
 - ☐ Charge de maintenance
 - ☐ Développeur
- Enregistrer les modifications** (button)
- Variables de projet**: Les variables peuvent être accidentellement exposées dans un job log ou envoyées à des fins malveillantes à un serveur tiers. La fonctionnalité de variable masquée peut aider à réduire le risque d'exposition accidentelle de valeurs variables, mais ne garantit pas que les utilisateurs malveillants n'accéderont pas aux variables. Comment rendre mes variables plus sûres ?
- Variables CI/CD**: A table with columns 'Clé', 'Valeur', 'Environnements', and 'Actions'. It currently shows 'Aucune variable pour le moment.' and an 'Ajouter une variable' button.
- Jetons de déclenchement de pipeline**: Déclenchez un pipeline pour une branche ou une étiquette en générant un jeton de déclenchement et en l'utilisant avec un appel d'API. Un emprunt d'identité donnera au jeton les permissions et l'accès au projet de l'utilisateur. En savoir plus.
- Gels de déploiement**: Ajoutez une période de gel pour empêcher la publication de releases non souhaitées pendant une certaine durée et pour un environnement donné. Vous devez actualiser les jobs de déploiement dans `.gitlab-ci.yml` pour refléter les gels de déploiement ajoutés ici. En savoir plus. Spécifiez les gels de déploiement avec la syntaxe cron.
- Permissions de jetons de job**

Blue arrows point to the 'Variables' section, the 'CI/CD' menu item, and the 'Ajouter une variable' button.

Description (optional)

Assignment - 8 | GitLab CI Project or Group Leve

La description de la valeur ou de l'utilisation de la variable.

Clé

Image_Name

Vous pouvez utiliser des variables CI/CD avec le même nom à différents endroits, mais ces variables peuvent s'écraser les unes les autres. Quel est l'ordre de précedence des variables ?

Valeur

1

Enregistrer les ...

Supprimer L...

An...

Mettre à jour le build_job et supprimez la section des variables que nous avons déclarées dans la dernière section Mettre à jour le script du build_job pour utiliser la variable définie « Image_Name » comme nom de notre image Docker.

The screenshot displays the GitLab CI/CD interface. At the top, there's a code editor showing a YAML pipeline configuration. The configuration includes two stages: 'pre_check' and 'build'. The 'pre_check' stage has a job 'pre_check_job' that runs 'docker:dind' and echoes some information. The 'build' stage has a job 'build_job' that runs 'docker:latest' and echoes the 'Image_Name' variable. A blue arrow points to the 'Image_Name' variable in the 'build_job' script. Below the code editor, there's a 'Message de commit' field with the text 'Assignment - 8 | GitLab CI Project or Group Level Variables'. The 'Branche' field is set to 'main'. At the bottom, there's a 'Pipeline' section showing the execution status of the pipeline. The pipeline is titled 'Assignment - 8 | GitLab CI Project or Group Level Variables' and is in a 'Succès' (Success) state. It shows three jobs: 'pre_check_job', 'build_job', and 'test_job', all of which are completed successfully. The 'build_job' job is highlighted with a green checkmark and a smiley face icon.

```
1 stages:
2   - pre_check
3   - build
4   - test
5
6 pre_check_job:
7   stage: pre_check
8   image: docker:stable
9   services:
10    - docker:dind
11   script:
12     - echo "Conteneur Docker en cours d'exécution"
13     - docker ps
14     - echo "Liste de toutes les images Docker disponibles :"

Message de commit  
Assignment - 8 | GitLab CI Project or Group Level Variables



Branche  
main



Valider les modifications Reinitialiser



Assignment - 8 | GitLab CI Project or Group Level Variables Supprimer



Succès Ulrich OGO a créé le pipeline pour la validation 1661a088. Il y a 4 minutes, terminé il y a 2 minutes



Pour main



dernier branche eo 3 jobs 2 2 minutes 0 seconde, mis en file d'attente pendant 1 secondes



Pipeline Jobs Tests



pre_check build test



pre_check_job build_job test_job


```

Section d'informations supplémentaires :

Dans GitLab CI/CD, vous pouvez définir des variables personnalisées au niveau du projet et du groupe, qui peuvent être utilisées dans vos pipelines CI/CD. Ces variables sont utiles pour stocker des informations sensibles telles que des clés API, des mots de passe ou toute autre donnée de configuration que vous ne souhaitez pas coder en dur dans vos scripts CI/CD ou vos fichiers de configuration.

Assignment - 9 | GitLab CI Pre Defined Variables

Configuration du pipeline CI GitLab pour les modifications ci-dessous Mettre à jour le script de build_job pour inclure une commande qui ajoute une balise à l'image Docker, qui est générée à l'aide de la commande docker build. Et utilise la variable d'environnement prédéfinie GitLab « \$CI_PIPELINE_IID » comme numéro de balise dans votre pipeline CI GitLab.

The image shows two screenshots from the GitLab interface. The top screenshot is the 'Catalogue CI/CD' editor for a file named '.gitlab-ci.yml'. It displays a YAML configuration with three stages: 'pre_check', 'build', and 'test'. The 'pre_check' stage has a job 'pre_check_job' using 'docker:stable' image and 'docker:dind' service. The 'build' stage has a job 'build_job' using 'docker:latest' image and 'docker:dind' service. The 'test' stage has a job 'test_job' using 'docker:stable' image and 'docker:dind' service. A blue arrow points to the 'build' stage script, specifically to the line: `docker build -t $Image_Name -t $Image_Name:$CI_PIPELINE_IID .`. Below the editor is the 'Message de commit' field containing 'Assignment - 9 | GitLab CI Pre Defined Variables' and the 'main' branch selected. The bottom screenshot shows the 'Assignment - 9 | GitLab CI Pre Defined Variables' pipeline execution view. It indicates the pipeline was created by 'Ulrich OGO' and is currently running. The pipeline status is 'En cours' (In progress) with 3 jobs. The jobs are: 'pre_check' (status: 'réussi' - success), 'build' (status: 'en cours' - in progress), and 'test' (status: 'réussi' - success). A 'Supprimer' (Delete) button is visible in the top right corner.

```
1 stages:
2   - pre_check
3   - build
4   - test
5
6 pre_check_job:
7   stage: pre_check
8   image: docker:stable
9   services:
10    - docker:dind
11   script:
12     - echo "Conteneur Docker en cours d'exécution"
13     - docker ps
14     - echo "Liste de toutes les images Docker disponibles :"

Message de commit  
Assignment - 9 | GitLab CI Pre Defined Variables



Branche  
main



Valider les modifications Reinitialiser



Assignment - 9 | GitLab CI Pre Defined Variables Supprimer



réussi Ulrich OGO a créé le pipeline pour la validation 26832586 Il y a 2 minutes, terminé il y a 14 secondes



Pour main



dernier branche 3 jobs 2.01 2 minutes 0 seconde, mis en file d'attente pendant 1 secondes



Pipeline Jobs 3 Tests 0



pre_check build test



pre_check_job build_job test_job


```

Section d'informations supplémentaires :

Section d'informations supplémentaires : \$CI_PIPELINE_IID- Dans GitLab, CI_PIPELINE_IID il s'agit d'une variable d'environnement qui contient l'identifiant unique du pipeline actuel. Cette variable est automatiquement définie par GitLab CI/CD et peut être utilisée dans vos fichiers de configuration CI/CD (comme .gitlab-ci.yml) ou dans vos scripts pour référencer le pipeline actuel.

Assignment - 10 | GitLab CI Pre Defined Variables

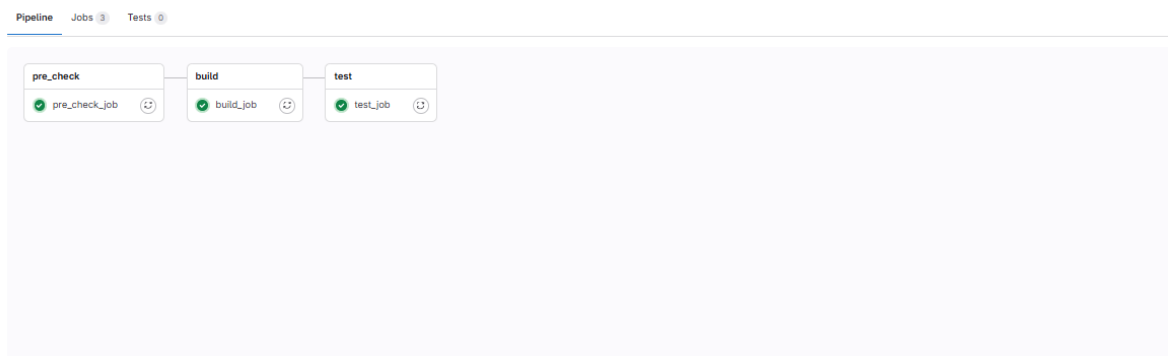
Configuration du pipeline CI GitLab pour les modifications ci-dessous Mettre à jour votre script de tâche CI_Build pour configurer le nom de l'image Docker comme nom de votre Project/Repo GitLab. Utilisez la variable d'environnement prédéfinie Gitlab « \$CI_PROJECT_NAME » comme nom de l'image Docker dans votre pipeline CI GitLab.



```
1 stages:
2   - pre_check
3   - build
4   - test
5
6 pre_check_job:
7   stage: pre_check
8   image: docker:stable
9   services:
10    - docker:dind
11   script:
12     - echo "Conteneur Docker en cours d'exécution"
13     - docker ps
14     - echo "Liste de toutes les images Docker disponibles :"
```

Assignment - 10 | GitLab CI Pre Defined Variables

Reuss Ulrich OGO a créé le pipeline pour la validation f648c088 Il y a 2 minutes, terminé il y a 38 secondes
Pour main
dernier branche 3 jobs 2.08 2 minutes 4 secondes, mis en file d'attente pendant 1 secondes



Section d'informations supplémentaires :

\$CI_PROJECT_NAME- Dans GitLab CI/CD, \$CI_PROJECT_NAME il s'agit d'une variable d'environnement qui contient le nom du projet associé au pipeline. Cette variable est automatiquement définie par GitLab CI/CD et peut être utilisée dans vos fichiers de configuration ou scripts CI/CD pour référencer le nom du projet de manière dynamique.

Assignment - 11 | GitLab CI Before Script

Configuration du pipeline CI GitLab pour les modifications ci-dessous Ajout du section `before_script`, sous `build_job`. Dans la section `before_script`, incluez la commande permettant d'imprimer la version de docker installée.

The image shows two screenshots from the GitLab interface. The top screenshot is the 'Catalogue CI/CD' editor for a file named 'Assignment - 11 | GitLab CI Before Script'. It displays a YAML configuration for a pipeline with two jobs: 'build_job' and 'test_job'. The 'build_job' stage is configured with 'image: docker:latest', 'services: - docker:dind #', and a 'before_script' section containing two commands: '- echo "Affichage de la version docker installé..."' and '- docker --version'. A blue arrow points to the 'before_script' section. The 'test_job' stage is configured with 'image: docker:latest', 'services: - docker:dind', and a 'script' section containing two commands: '- echo "Afficher tous les conteneurs"' and '- docker ps -a'. Below the configuration, the 'Message de commit' field contains the filename, and the 'Branche' field is set to 'main'. The bottom screenshot shows the 'Pipeline' view for the same pipeline. It indicates that the pipeline was successfully executed ('Reussit') by 'Ulrich OGO' on the 'main' branch. The pipeline consists of three jobs: 'pre_check' (status: success), 'build' (status: success), and 'test' (status: success). The 'build' job is highlighted, showing its duration of 2.09 minutes and 52 seconds.

```
13 - docker ps
14 - echo "Liste de toutes les images Docker disponibles :"
```

```
15 - docker images
16
17 build_job:
18   stage: build
19   image: docker:latest
20   services:
21     - docker:dind #
22   before_script:
23     - echo "Affichage de la version docker installé..."
24     - docker --version
25   script:
26     - echo "Construction de l'image Docker"
27     - docker build -t $Image_Name -t $CI_PROJECT_NAME:$CI_PIPELINE_IID .
28     - echo "Image $Image_Name et $CI_PIPELINE_IID sont construites avec succès a comme nom $CI_PROJECT_NAME"
29
30 test_job:
31   stage: test
32   image: docker:latest
33   services:
34     - docker:dind
35   script:
36     - echo "Afficher tous les conteneurs"
37     - docker ps -a
38
```

Message de commit

Assignment - 11 | GitLab CI Before Script

Branche

main

Valider les modifications Reinitialiser

Assignment - 11 | GitLab CI Before Script Supprimer

Reussit Ulrich OGO a créé le pipeline pour la validation 67f6581f. Il y a 3 minutes, terminé il y a 52 secondes

Pour main

3 jobs 2.09 2 minutes 5 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs Tests

pre_check build test

pre_check_job build_job test_job

Assignment- 12 | GitLab CI After Script

Configuration du pipeline CI GitLab pour les modifications ci-dessous Ajoute la section `after_script` sous `build_job` Ajoute une commande sous la section `after_script` pour répertorier toutes les images Docker.

Catalogue CI/CD

Aide

```
7  stage: pre_check
8  image: docker:stable
9  services:
10 |   - docker:dind
11
12  script:
13 |   - echo "Conteneur Docker en cours d'exécution"
14 |   - docker ps
15 |   - echo "Liste de toutes les images Docker disponibles ::"
16 |   - docker images
17
18  build_job:
19  stage: build
20  image: docker:latest
21  services:
22 |   - docker:dind #
23
24  before_script:
25 |   - echo "Affichage de la version docker installé..."
26 |   - docker --version
27
28  script:
29 |   - echo "Construction de l'image Docker"
30 |   - docker build -t $Image_Name -t $CI_PROJECT_NAME:$CI_PIPELINE_IID .
31 |   - echo "Image $Image_Name et $CI_PIPELINE_IID sont construites avec succès a comme nom $CI_PROJECT_NAME"
32
33  after_script:
34 |   - echo "Cmd pour regénérer toutes les images Docker"
35 |   - docker images
```

Message de commit

Assignment - 12 | GitLab CI After Script

Branche

main

Valider les modifications

Reinitialiser

Assignment - 12 | GitLab CI After Script

Supprimer

🟢 Réussi Ulrich OGO a créé le pipeline pour la validation 9156cbab 🕒 Il y a 4 minutes, terminé il y a 2 minutes

Pour [main](#)

🟢 dernier [branche](#) 🔄 3 jobs ⌚ 2.04 🕒 2 minutes 2 secondes, mis en file d'attente pendant 1 secondes

Pipeline

Jobs 3

Tests 0

pre_check

build

test

🟢 pre_check_job 😊

🟢 build_job 😊

🟢 test_job 😊

Assignment - 13 | Dockerfile Setup for parallel builds

Configuration du pipeline CI GitLab pour les modifications ci-dessous Nous allons créer un répertoire au niveau racine dans la branche principale du projet GitLab et nommez le répertoire « api ». Dans le répertoire « api », créez un Dockerfile et ajoutez le contenu ci-dessous à l'intérieur du fichier.

Valider les modifications

Nom du répertoire

api

Message de commit

repertoire api

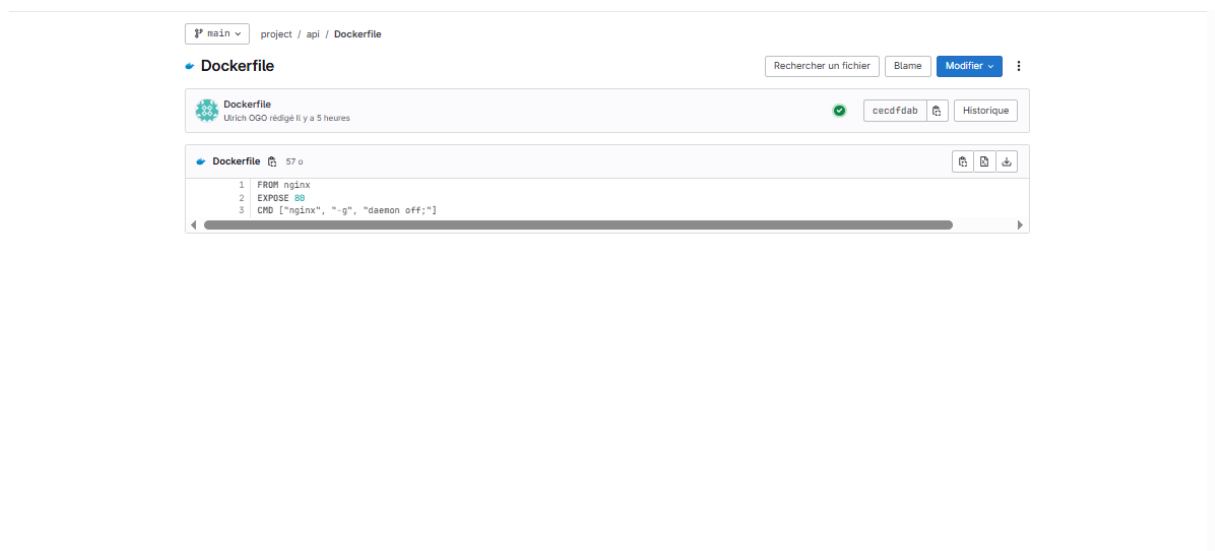
Branche

☒ Valider dans la branche **main** actuelle

☐ Valider dans une nouvelle branche

Annuler

Valider les modifications



Assignment - 14 | GitLab CI Multiple Jobs under same Stage

Configuration du pipeline CI GitLab pour les modifications ci-dessous Ajouter une nouvelle tâche « build_job_api » et définir le nom de l'étape sur « build ». Configure « build_job_api » avec des balises. Configure « build_job_api » pour créer le Dockerfile placé sous le répertoire « api ». Définir le nom de l'image Docker sur « docker-api » et utilisez uniquement la dernière balise.

Le premier code après de nombres erreur nous avons réussi a exécuté le job sans erreur En donnant : le tag du Runner créer depuis le début de la série d'exercice DOKER_HOST #Point Docker vers le service docker:dind DOCKER_DRIVER : overlay2 # Utilisez le pilote de stockage overlay2 DOCKER_TLS_CERTDIR : communiquer en mode non sécurisé avec le démon Docker-in Docker Optimisation du code, nous allons enregistrer les variables dans section variables du paramètre

Description (optional)

pilotage du stockage

La description de la valeur ou de l'utilisation de la variable.

Clé

DOCKER_DRIVER

Vous pouvez utiliser des variables CI/CD avec le même nom à différents endroits, mais ces variables peuvent s'écraser les unes les autres. Quel est l'ordre de précedence des variables ?

Valeur

over lay2

Enregistrer les ...

Supprimer L...

An...

Description (optional)

connexion

La description de la valeur ou de l'utilisation de la variable.

Clé

DOCKER_HOST

Vous pouvez utiliser des variables CI/CD avec le même nom à différents endroits, mais ces variables peuvent s'écraser les unes les autres. Quel est l'ordre de précedence des variables ?

Valeur

tcp://docker:2375

La valeur de la variable sera évaluée en tant que chaîne brute.

Ajouter une variable

Annuler

[Catalogue CI/CD](#) [Aide](#)

```
30 | - echo "Cmd pour repertorier toutes les images Docker"
31 | - docker images
32 |
33 | build_job_api:
34 |   stage: build
35 |   image: docker:latest
36 |   services:
37 |     - docker:dind
38 |   tags:
39 |     - project
40 |   variables:
41 |     DOCKER_TLS_CERTDIR: ""
42 |   script:
43 |     - echo "Création de l'image Docker pour l'API..."
44 |     - docker build -t docker-api:latest -f api/Dockerfile api
45 |     - echo "Image Docker pour l'API créé avec succès."
46 |
47 | test_job:
48 |   stage: test
49 |   image: docker:latest
50 |   services:
51 |     - docker:dind
52 |   script:
53 |     - echo "Afficher tous les conteneurs"
54 |     - docker ps -a
55 |
```

Message de commit
Assignment - 14 | GitLab CI Multiple Jobs under same Stage

Branche
main

[Valider les modifications](#) [Reinitialiser](#)

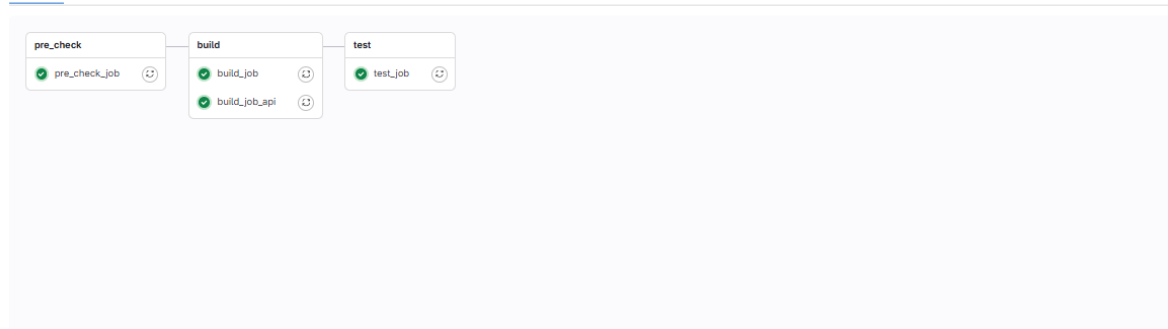
Assignment - 14 | GitLab CI Multiple Jobs under same Stage

Reussir Ulrich OGO a créé le pipeline pour la validation 492ba1f5 il y a 3 minutes, terminé il y a 1 minute

Pour main

dernier branche 4 jobs 2.03 2 minutes 19 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs 4 Tests 0



Assignment - 15 | GitLab CI Job Failures

Configuration du pipeline CI GitLab pour les modifications ci-dessous Mettre à jour la section script de la tâche pre_check_job pour inclure la commande supplémentaire « dummy ». Valider et transmettre les modifications pour déclencher le pipeline CI GitLab. Et surveille le pipeline pour observer l'échec causé par la commande « dummy ». Commentez !

[Catalogue CI/CD](#) [Aide](#)

```
1 stages:
2   - pre_check
3   - build
4   - test
5
6 pre_check_job:
7   stage: pre_check
8   image: docker:stable
9   services:
10    - docker:dind
11   script:
12     - dummy
13     - echo "Conteneur Docker en cours d'exécution"
14     - docker ps
15     - echo "Liste de toutes les images Docker disponibles :"

Message de commit  
Assignment - 15 | GitLab CI Job Failures



Branche  
main



Valider les modifications Reinitialiser


```

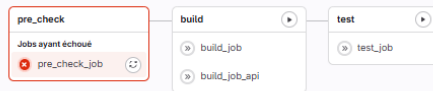
Assignment - 15 | GitLab CI Job Failures

En échec Ulrich OGO a créé le pipeline pour la validation 15c82b4c Il y a 8 minutes, terminé Il y a 7 minutes

Pour main

dernier branche 4 jobs 0.71 42 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs 4 Jobs ayant échoué 1 Tests 0



```
20 Executing "step_script" stage of the job script
21 Using docker image sha256:b0757c55a1fdbb59c378fd34dde3e12bd25f68094dd69546cf5ca80ddbae7a33 for docker:stable with digest docker@sha256:fd4d028713fd05a1fb896412885daed82c4
a8cc84331d8dad80cb596d7ce3e3a ...
22 /bin/sh: eval: line 154: dummy: not found
23 $ dummy
```

Assignment - 16 | GitLab CI Allow Failures

Configurez votre pipeline GitLab CI pour les modifications ci-dessous : Nous avons donc mis à jour la section Script de pre_check_job et ajouté une commande supplémentaire « dummy » dans la question précédente, ce qui a provoqué l'échec du pipeline. Mais disons que nous acceptons tout échec dans le pre_check_job et que nous souhaitons poursuivre l'exécution du pipeline même si le pre_check_job échoue. Nous allons essayer donc de gérer l'échec dans votre pre_check_job, même si pre_check_job échoue, le pipeline continuera de s'exécuter.

```
pre_check_job:
  stage: pre_check
  image: docker:stable
  allow_failure: true
  services:
    - docker:dind
  script:
    - dummy
    - echo "Vérification des conteneurs Docker en cours..."
    - docker ps
    - echo "Lister toutes les images Docker :"
    - docker images
```

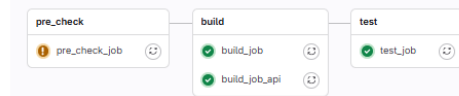
Assignment - 16 | GitLab CI Allow Failures

Avertissement Ulrich OGO a créé le pipeline pour la validation 3f495883 Il y a 2 minutes, terminé il y a 28 secondes

Pour **main**

dernier branche 4 jobs 2.00 2 minutes 20 secondes, mis en file d'attente pendant 0 secondes

Pipeline Jobs 4 Jobs ayant échoué 1 Tests 0



Section d'informations supplémentaires :

Dans GitLab CI/CD, vous pouvez marquer un travail comme autorisant l'échec à l'aide du mot-clé `allow_failure`. Cela est utile lorsque vous avez des tâches qui ne sont pas essentielles à la réussite du pipeline, telles que des tâches de test ou de déploiement facultatives, et que vous souhaitez que le pipeline continue même si ces tâches échouent.

Assignment - 17 | GitLab CI Concurrent Limits

Configurez votre pipeline GitLab CI pour les modifications ci-dessous : Nous allons mettre à jour la configuration de notre machine Runner à 6, afin que nous puissions exécuter 6 tâches en parallèle En modifiant le fichier `/etc/gitlab-runner/config.toml`

```
concurrent = 6
check_interval = 0
connection_max_age = "15m0s"
shutdown_timeout = 0
```

`concurrent = 6`

`gitlab-runner restart`

```
root@testing:/etc/gitlab-runner# vim config.toml
root@testing:/etc/gitlab-runner#
root@testing:/etc/gitlab-runner# gitlab-runner restart
Runtime platform arch=amd64 os=linux pid=22577 revision=0f67ff19 version=17.11.0
root@testing:/etc/gitlab-runner#
root@testing:/etc/gitlab-runner#
```

Assignment - 18 | GitLab CI Rules

Configure le pipeline GitLab CI pour les modifications ci-dessous : Supprimer la commande « dummy » de la section Script de `pre_check_job`. Configurer le pipeline GitLab CI `build_job` pour qu'il soit déclenché uniquement si le nom de la branche est « main » et qu'aucune autre branche ne peut déclencher le pipeline. Avec règles : `'$CI_COMMIT_BRANCH == "main"'`

Catalogue CI/CD Aide

```

9      allow_failure: true
10     services:
11       - docker:dind
12     script:
13       - echo "Conteneur Docker en cours d'exécution"
14       - docker ps
15       - echo "Liste de toutes les images Docker disponibles :"

```

Message de commit

Assignment - 18 | GitLab CI Rules

Branche

main

Assignment - 18 | GitLab CI Rules

Ulrich OGO a créé le pipeline pour la validation (a37be63) 11 y a 2 minutes, terminé à l'instant

Pour main

4 jobs 2:17 2 minutes 31 secondes, mis en file d'attente pendant 1 secondes

Pipeline

Jobs Tests

pre_check

build

test

Assignment - 19 | GitLab CI Needs

Configurer les tâches pre_check_job, build_job et build_job_api du pipeline CI GitLab pour qu'elles s'exécutent en parallèle. Pour l'exécution des jobs en parallèle à la fin de chaque job nous mettre le needs : []

Catalogue CI/CD Aide

```

1      stages:
2        - pre_check
3        - build
4        - test
5
6     pre_check_job:
7       stage: pre_check
8       image: docker:stable
9       allow_failure: true
10      services:
11        - docker:dind
12      script:
13        - echo "Conteneur Docker en cours d'exécution"
14        - docker ps
15        - echo "Liste de toutes les images Docker disponibles :"

```

Message de commit

Assignment - 19 | GitLab CI Needs

Branche

main

Valider les modifications

Reinitialiser

Assignment - 19 | GitLab CI Needs

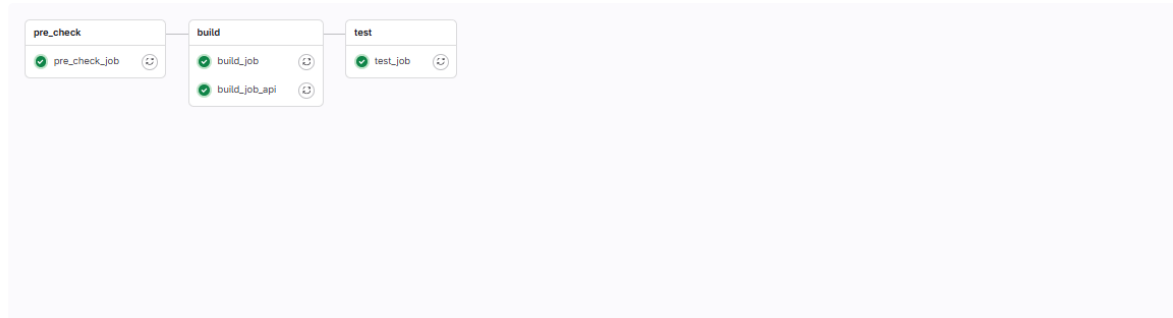
✓ Reussi Ulrich OGO a créé le pipeline pour la validation 33b7ff38 🗄 Il y a 2 minutes, terminé Il y a 33 secondes

Pour **main**

dernier | branche 4 jobs 2.02 2 minutes 21 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs 4 Tests 0

Grouper les jobs par Etape Dépendances des jobs



Assignment - 20 | GitLab CI Needs

Configure le pipeline CI GitLab pour les modifications ci-dessous : Configure la tâche « test_job » du pipeline CI GitLab de sorte qu'elle doive attendre que « build_job_api » soit terminé. Ainsi, notre « test_job » sera déclenché une fois « build_job_api » terminé.

```
35 | - if: $CI_COMMIT_BRANCH == "main"
36 | build_job_api:
37 |   stage: build
38 |   image: docker:latest
39 |   services:
40 |     - docker:dind
41 |   tags:
42 |     - project
43 |   variables:
44 |     DOCKER_TLS_CERTDIR: ""
45 |   script:
46 |     - echo "Création de l'image Docker pour l'API..."
47 |     - docker build -t docker-api:latest -f api/Dockerfile api
48 |     - echo "Image Docker pour l'API créé avec succès."
49 |
50 | test_job:
51 |   stage: test
52 |   image: docker:latest
53 |   services:
54 |     - docker:dind
55 |   script:
56 |     - echo "Afficher tous les conteneurs"
57 |     - docker ps -a
58 |   needs:
59 |     - build_job_api
```

Message de commit

Assignment - 20 | GitLab CI Needs

Branche

main

Valider les modifications

Réinitialiser



Assignment - 20 | GitLab CI Needs

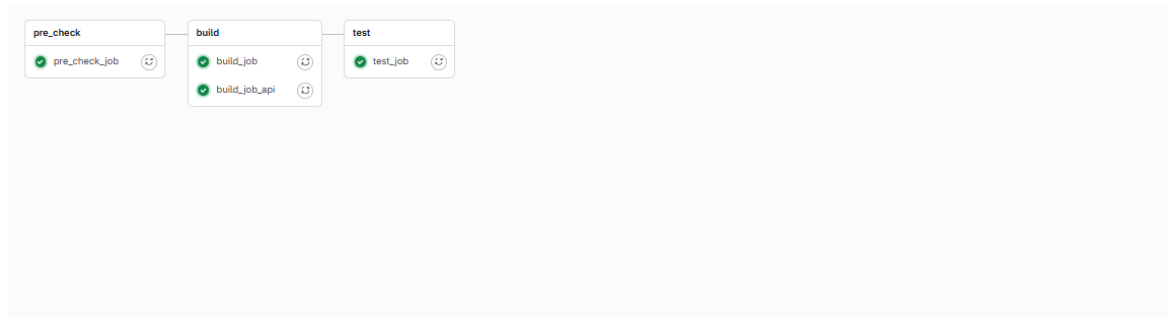
Reussir Ulrich OGO a créé le pipeline pour la validation @e0123568 il y a 2 minutes, terminé à l'instant

Pour main

dernier branche 4 jobs 2.00 2 minutes 19 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs 4 Tests 0

Grouper les jobs par Etape Dépendances des jobs



Assignment - 21 | GitLab CI Retry

Configure le pipeline CI GitLab pour les modifications ci-dessous : Mettre à jour la section before_script de build_job et ajoutez une commande « dummy » Étant donné qu'il n'existe pas de commande « dummy » sous Linux, notre build_job échouera. Mais nous voulons que notre build_job réessaye au moins 2 fois avant d'échouer.

Catalogue CI/CD Aide

```
19 build_job:
20   stage: build
21   image: docker:latest
22   services:
23     - docker:dind #
24   before_script:
25     - echo "Affichage de la version docker installé..."
26     - docker --version
27     - dummy
28   script:
29     - echo "Construction de l'image Docker"
30     - docker build -t $Image_Name -t $CI_PROJECT_NAME:$CI_PIPELINE_IID .
31     - echo "Image $Image_Name et $CI_PIPELINE_IID sont construites avec succès a comme nom $CI_PROJECT_NAME"
32   after_script:
33     - echo "Cmd pour repertorier toutes les images Docker"
34     - docker images
35   rules:
36     - if: $CI_COMMIT_BRANCH == "main"
37   needs: []
38   retry: 2
39
40 build_job_api:
41   stage: build
42   image: docker:latest
43   services:
44     - docker:dind
```

Message de commit

Assignment - 21 | GitLab CI Retry

Branche

main

Valider les modifications Reinitialiser

Assignment - 21 | GitLab CI Retry

En échec Ulrich OGO a créé le pipeline pour la validation `abde93e` Il y a 2 minutes, terminé à l'instant

Pour `main`

dernier branche 6 jobs 3.31 2 minutes 23 secondes, mis en file d'attente pendant 2 secondes

Pipeline Jobs Tests

Grouper les jobs par Etape Dépendances des jobs

Assignment - 22 | GitLab CI Timeout

Configure le pipeline CI GitLab pour les modifications ci-dessous : Mettre à jour le `build_job` et supprimez la commande « `dummy` » de la section `before_script`. Mettre à jour le `build_job` et ajoutez un délai d'attente de 10 secondes dans le travail. Le travail échouera en raison d'une valeur de délai d'attente trop faible. Pour le faire nous utilisons `sleep 10`

Catalogue CI/CD Aide

```

19 build_job:
20   stage: build
21   image: docker:latest
22   services:
23     - docker:dind #
24   before_script:
25     - echo "Affichage de la version docker installé..."
26     - docker --version
27   script:
28     - echo "Construction de l'image Docker"
29     - docker build -t $Image_Name -t $CI_PROJECT_NAME:$CI_PIPELINE_ID .
30     - echo "Image $Image_Name et $CI_PIPELINE_ID sont construites avec succès a comme nom $CI_PROJECT_NAME"
31     - sleep 10
32   after_script:
33     - echo "Cmd pour repertoirer toutes les images Docker"
34     - docker images
35   rules:
36     - if: $CI_COMMIT_BRANCH == "main"
37   needs: []
38   retry: 2
39
40 build_job_api:
41   stage: build
42   image: docker:latest
43   services:
44     - docker:dind
  
```

Message de commit

Assignment - 22 | GitLab CI Timeout

Branche

main

Valider les modifications Reinitialiser

Assignment - 22 | GitLab CI Timeout

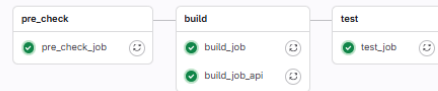
✓ Reussir Ulrich OGO a créé le pipeline pour la validation 388a6768 il y a 5 minutes, terminé il y a 3 minutes

Pour main

dernier branche 4 jobs 2.16 2 minutes 29 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs 4 Tests 0

Grouper les jobs par Etape Dépendances des jobs



Assignment - 23 | GitLab CI Default

Configure le pipeline CI GitLab pour les modifications ci-dessous : Supprimez le délai d'expiration que nous avons ajouté à l'intérieur de la tâche à 10 secondes. À l'aide du mot-clé default, définissez le délai d'expiration à 10 minutes au niveau du pipeline. Default : timeout :10

[Catalogue CI/CD](#) [Aide](#)

```
1 default:
2   timeout: 10m
3   stages:
4     - pre_check
5     - build
6     - test
7
8   pre_check_job:
9     stage: pre_check
10    image: docker:stable
11    allow_failure: true
12    services:
13      - docker:dind
14    script:
15      - echo "Conteneur Docker en cours d'exécution"
16      - docker ps
17      - echo "Liste de toutes les images Docker disponibles :"

Message de commit  
Assignment - 23 | GitLab CI Default



Branche  
main



Valider les modifications Reinitialiser


```



Assignment - 23 | GitLab CI Default

[Supprimer](#)

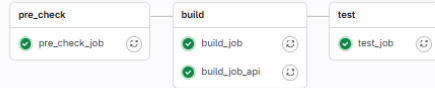
✓ Reussit Ulrich OGO a créé le pipeline pour la validation 36359c09 il y a 3 minutes, terminé il y a 40 secondes

Pour [main](#)

derrière [branche](#) 4 jobs 2.26 2 minutes 31 secondes, mis en file d'attente pendant 2 secondes

Pipeline Jobs 4 Tests 0

Grouper les jobs par Etape Dépendances des jobs



Assignment - 24 | GitLab CI When

Configure le pipeline CI GitLab pour les modifications ci-dessous : une approbation manuelle sur test_job, de sorte que lorsque nous approuverons cette tâche, elle sera déclenchée dans notre pipeline.

[Catalogue CI/CD](#) [Aide](#)

```
42 build_job_api:
43   stage: build
44   image: docker:latest
45   services:
46     - docker:dind
47   tags:
48     - project
49   variables:
50     DOCKER_TLS_CERTDIR: ""
51   script:
52     - echo "Création de l'image Docker pour l'API..."
53     - docker build -t docker-api:latest -f api/Dockerfile api
54     - echo "Image Docker pour l'API créée avec succès."
55
56 test_job:
57   stage: test
58   image: docker:latest
59   services:
60     - docker:dind
61   script:
62     - echo "Afficher tous les conteneurs"
63     - docker ps -a
64   needs:
65     - build_job_api
66   when: manual
67
```

Message de commit

Assignment - 24 | GitLab CI When

Branche

main

[Valider les modifications](#) [Reinitialiser](#)

test_job

Manuel Créé il y a 3 minutes par Ulrich OGO



Ce job nécessite une action manuelle

Ce job ne parvient pas à démarrer automatiquement et doit être lancé manuellement.

[Exécuter le job](#)

Source: Push

Validation 84566e97

Assignment - 24 | GitLab CI When

Pipeline #1797899901 ✓ Reussit pour main

test

Tâches liées

→ test_job

Assignment - 24 | GitLab CI When

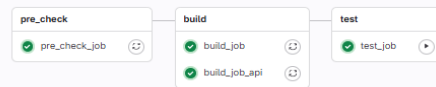
Reuss Ulrich OGO a créé le pipeline pour la validation 84566e97. Il y a 5 minutes, terminé il y a 28 secondes

Pour main

Reuss branche 4 jobs 2.21 2 minutes 40 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs 4 Tests 0

Grouper les jobs par Etape Dépendances des jobs



Assignment - 25 | GitLab CI Parallel

Configure le pipeline CI GitLab pour les modifications ci-dessous : Exécute build_job 2 fois en parallèle en même temps. À l'aide du mot-clé parallel, définissez la limite sur 2. Pour exécuter la tâche build_job deux fois en parallèle dans GitLab CI/CD, vous pouvez utiliser le mot-clé parallel pour définir une limite de 2 parallel :

Catalogue CI/CD Aide

```
16 - docker ps
17 - echo "Liste de toutes les images Docker disponibles :"

Message de commit



Assignment - 25 | GitLab CI Parallel



Branche



main



Valider les modifications Réinitialiser


```

Assignment - 25 | GitLab CI Parallel

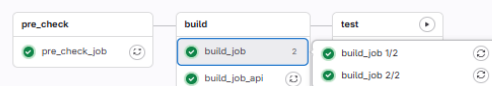
Reuss Ulrich OGO a créé le pipeline pour la validation b6b9f1c3. Il y a 3 minutes, terminé il y a 1 minute

Pour main

Reuss branche 5 jobs 2.5 2 minutes 7 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs 5 Tests 0

Grouper les jobs par Etape Dépendances des jobs



Assignment - 25 | GitLab CI Parallel

[Supprimer](#)

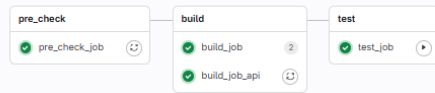
✓ Reussir Ulrich OGO a créé le pipeline pour la validation `33b9ff1c2` il y a 5 minutes, terminé il y a 15 secondes

Pour `main`

dernier branche 5 jobs 3.13 2 minutes 44 secondes, mis en file d'attente pendant 1 secondes

Pipeline Jobs 5 Tests 0

Grouper les jobs par Etape Dépendances des jobs



FIN DU TP